

Full Stack Development with MERN

Project Documentation Format

1. Introduction

- **Project Title:** Smart Library Management System
- **Team Members:** Aravinth. P. (Lead Developer), Team Member 2 (Backend Developer), Team Member 3 (Frontend Developer)

2. Project Overview

- **Purpose:** The purpose of this project is to automate the daily operations of a library, including book inventory management, member registration, tracking of issued and returned books, and calculating late fines efficiently.
- **Features:** Role-based access (Admin/Member), real-time book search and filtering, automated email reminders, dashboard analytics, and fine management.

3. Architecture

- **Frontend:** Built using React.js for a dynamic Single Page Application (SPA). State management is handled by Redux Toolkit, and UI components are styled using Tailwind CSS.
- **Backend:** Developed using Node.js and Express.js to create robust RESTful APIs for handling client requests and business logic.
- **Database:** MongoDB serves as the NoSQL database, structured using Mongoose ODM with collections for Users, Books, and BorrowTransactions.

4. Setup Instructions

- **Prerequisites:** Node.js (v16+), MongoDB (Local or Atlas URI), Git.
- **Installation:**
 1. Clone the repository: `git clone https://github.com/team/library-system.git`
 2. Setup Backend: `cd server -> npm install -> create .env (MONGO_URI, JWT_SECRET)`
 3. Setup Frontend: `cd client -> npm install -> create .env (REACT_APP_API_URL)`

5. Folder Structure

- **Client:** /src contains /components (reusable UI), /pages (main views), /redux (state), and /assets.
- **Server:** /src contains /models (Mongoose schemas), /controllers (API logic), /routes (endpoints), and /middleware.

6. Running the Application

Provide commands to start the frontend and backend servers locally:

- **Frontend:** `npm start` in the client directory.
- **Backend:** `npm start` or `npm run dev` in the server directory.

7. API Documentation

Document all endpoints exposed by the backend:

- **Authentication:** `POST /api/auth/register`, `POST /api/auth/login`
- **Books:** `GET /api/books` (Fetch all), `POST /api/books/issue` (Issue book).

8. Authentication

Authentication is handled using JSON Web Tokens (JWT). Upon login, the server issues a token which the client stores in localStorage. Protected routes require this token in the Authorization header. Passwords are encrypted using bcrypt.

9. User Interface

[Insert Screenshot - Login Page]

[Insert Screenshot - Admin Dashboard]

10. Testing

Backend APIs are tested using Jest and Supertest. Frontend components are tested using React Testing Library.

11. Screenshots or Demo

- **Demo Link:** <https://smart-library-demo.herokuapp.com>
- **GitHub Repo:** <https://github.com/team-lib/smart-library>

12. Known Issues

Session expiration does not currently auto-redirect the user to the login page immediately.

13. Future Enhancements

Integration of a payment gateway (e.g., Stripe) to collect late fines directly through the portal. Implementation of an AI-based book recommendation system.